CPE 314 — Software Design

University of Cebu

Computer Engineering Department

Plate N Platter

Plan smarter. Eat better. Cook easier.

Group Name / No.:	PNP - 1
Members:	Kirk Quisagan
	Ken Tamidles
	Kimberly Luistro
	Yvonne Mae Tapan
	Venice Nadela
	Nesley Cate Librea
Instructor:	Richyll Son
Term:	First Semester, S.Y. 2026-2027
Document Version / Date:	0.1 / October 22,2024

Table of Contents

Revision History

Updated each week as the team adds or revises sections of this document.

Table 1: Revision history

Version	Date	Author	Changes
0.1	October 22, 2026	Group PNP	Initial draft: Week 8 sections (1 and 2), plus early drafts of Sections 3–9
0.2			Week 9 sections added
0.3			Week 10 sections added
1.0			Final compilation and polish — Week 11

1. Introduction & Project Overview

Plate N Platter is a personal meal-planning desktop application for individual users such as students and busy households, built for Option H (Recipe & Meal Planner) of CPE 314. Users register, log in, and plan breakfast, lunch, dinner, and snacks on a calendar where every meal is color-coded by Healthy Level (Healthy, Moderate, or Indulgent) and labeled Quick, Easy, or Hard to prepare. Recipes can be imported from TheMealDB (including Filipino dishes) or created by the user, can be built from other recipes, and feed an automatic shopping list, a cook-next queue, an undo function, and a daily streak that rewards eating at least one healthy meal a day. The app solves the everyday problem of deciding what to cook, knowing what to buy, and staying consistent with healthier eating, all in one simple tool.

Table 2: Project summary

Item	Details
Theme	Option H — Recipe & Meal Planner
Target users	Individual users (students, busy households); each user has a private account
Platform	Java 17+ desktop application with a JavaFX graphical interface
Data storage	SQLite database accessed through JDBC using PreparedStatement
Public API	TheMealDB (https://www.themealdb.com/api.php), shared test key “1”
Key features	Register/login, calendar with Healthy Level colors, today’s meals with photos, recipe browsing and creation, nested recipes, cook queue, undo, shopping list, streak and badges, weekly statistics with a bar chart

2. Preliminary ER Sketch & Team Roles

The sketch below comes from our Week 8 proposal. It lists the entities and their key attributes. It will be superseded by the finalized ERD in Section 4.

```
User(id PK, name, username UNIQUE, age, gender, passwordHash, salt)
Recipe(id PK, userId FK, name, mealType, healthyLevel, cuisine, instructions,
       prepTimeMinutes, servings, calories, protein, carbs, fat, sugar,
       imageUrl, externalId)
Ingredient(id PK, name)
RecipeIngredient(recipeId FK, ingredientId FK, measure)
RecipeComponent(parentRecipeId FK, subRecipeId FK, quantity)
MealPlanEntry(id PK, userId FK, planDate, mealSlot, recipeId FK, eaten)
ShoppingListItem(id PK, userId FK, ingredientId FK, quantity, checked)
CookQueueEntry(id PK, userId FK, recipeId FK, addedAt)
Streak(userId PK/FK, currentStreak, bestStreak, cheatDaysAvailable, lastHealthyDay)
UserBadge(id PK, userId FK, badgeName, dateEarned)
Favorite(userId FK, recipeId FK)            -- stretch feature
```

Figure 1: Preliminary ER sketch in text notation (entities and key attributes)

Table 3: Relationships in the preliminary sketch

From	To	Cardinality	Meaning
User	Recipe	1 : M	A user creates many recipes
User	MealPlanEntry	1 : M	A user plans many meals on the calendar
Recipe	MealPlanEntry	1 : M	A recipe can be planned many times
Recipe	Ingredient	M : N	Linked through RecipeIngredient
Recipe	Recipe	M : N	A recipe uses sub-recipes, linked through RecipeComponent
User	ShoppingListItem	1 : M	Each user has a shopping list
User	CookQueueEntry	1 : M	Each user has a cook queue
User	Streak	1 : 1	One streak record per user
User	UserBadge	1 : M	A user earns many badges
User	Recipe (Favorite)	M : N	A user saves favorite recipes (stretch)

Table 4: Team members and primary roles

Member	Primary role
Kirk Quisagan	SQLite schema and JDBC data-access classes
Ken Tamidles	TheMealDB API client and JSON parsing (plus the nutrition API if adopted)
Kimberly Luistro	Object-oriented classes (Recipe hierarchy, MealPlanEntry), cooking queue, and undo stack
Yvonne Mae Tapan	JavaFX login/register screens and Home layout
Venice Nadela	Calendar grid and Meal Times screen
Nesley Cate Librea	Recursion, JUnit tests, statistics, streak tracking, and chart

3. System Architecture & Object-Oriented Design

Status: early draft (Week 9 section). Based on the current starter code.

The design centers on the abstract class Recipe. It holds the data every dish shares and declares three methods that each concrete subclass answers differently: the meal type, a base preparation time, and extra minutes per ingredient. The same call to estimateOwnPrepMinutes() therefore gives a breakfast, a main course, and a dessert different results (polymorphism).

```
Recipe (abstract)
  +-- BreakfastRecipe    base 5 min  + 1 min per ingredient
  +-- MainCourseRecipe   base 15 min + 3 min per ingredient
  +-- DessertRecipe      base 10 min + 2 min per ingredient

A Recipe may also hold a list of sub-recipes (a Recipe uses other Recipes).
```

Figure 2: Recipe class hierarchy

Table 5: Classes and their single responsibility

Class	Single responsibility
Recipe and its three subclasses	Hold one dish's data and estimate its own prep time
RecipeFactory	Create the right Recipe subclass from a meal-type text
RecipeCalculator	Recursive totals of prep time, calories, and ingredients; difficulty; circular-link check
UndoManager / UndoableAction	Stack of reversible actions (last in, first out)
CookQueue	First-in-first-out queue of recipes to cook
ShoppingListBuilder	Merge ingredients of planned recipes into one map
StreakCalculator	Count consecutive healthy days
PasswordUtil	Salted password hashing and verification
Database	Open connections and create the tables
UserDao, RecipeDao	Read and write one table group each using PreparedStatement
MealDbClient	Call TheMealDB and convert the JSON into Recipe objects
LoginView, MainView, App	Display screens and handle user events only (no business logic)

Design rationale: business rules (calculations, queue, undo, streak) live in plain Java classes that know nothing about JavaFX or SQL. This keeps each class focused on one job, lets the rules be tested with JUnit without opening a window or a database, and lets teammates work on screens, data access, and logic in parallel without editing the same files.

4. Data Model & Database Design

Status: early draft (Week 9 section). Matches the Database class of the starter code.

The ER diagram has 11 entities and the relationships shown in Figure 3. Primary keys are highlighted in yellow and foreign keys in blue. Cardinalities follow the listing in Table 3. *Changes may occur after the proposal*

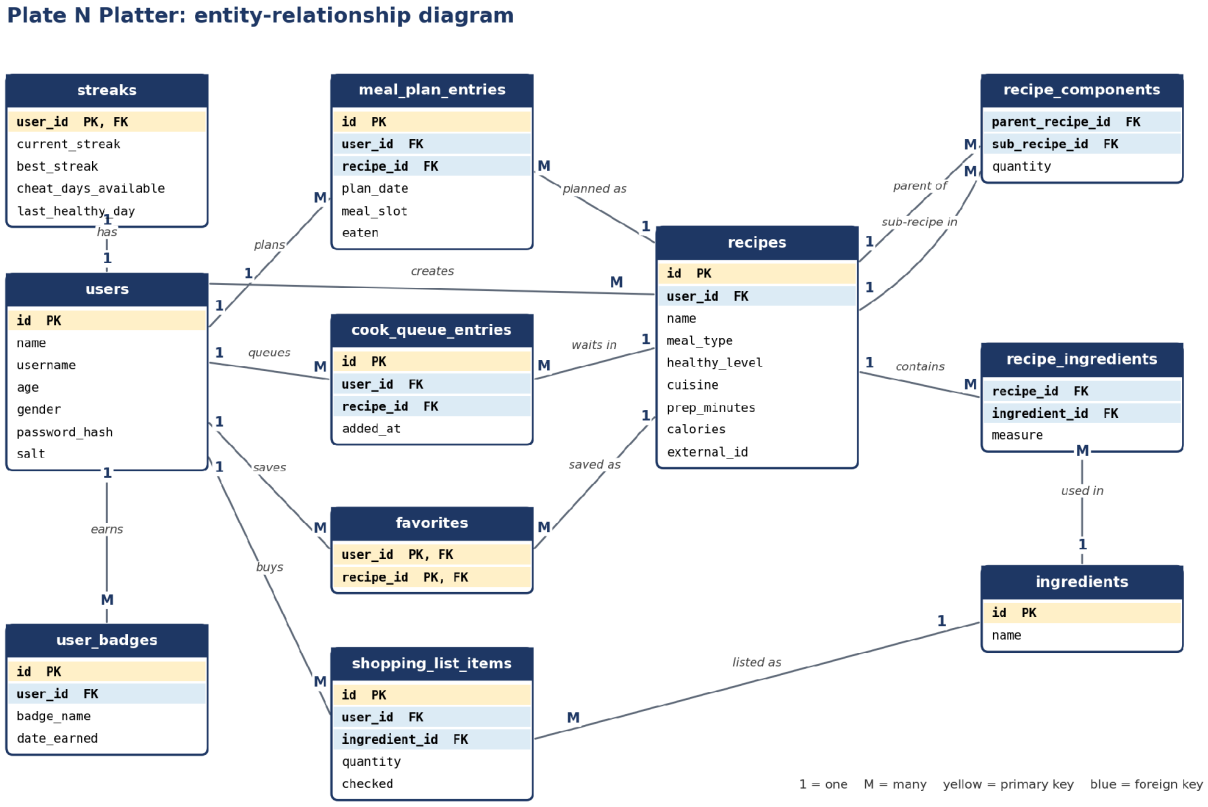


Figure 3: Final entity-relationship diagram (key columns shown)

Table 6: SQLite schema (all columns)

Table	Columns
users	id (PK), name, username (UNIQUE), age, gender, password_hash, salt
recipes	id (PK), user_id (FK users), name, meal_type, healthy_level, cuisine, instructions, prep_minutes, servings, calories, protein, carbs, fat, sugar, image_url, external_id
ingredients	id (PK), name (UNIQUE)
recipe_ingredients	recipe_id (FK recipes, ON DELETE CASCADE), ingredient_id (FK ingredients), measure
recipe_components	parent_recipe_id (FK recipes), sub_recipe_id (FK recipes), quantity
meal_plan_entries	id (PK), user_id (FK), plan_date, meal_slot, recipe_id (FK), eaten
shopping_list_items	id (PK), user_id (FK), ingredient_id (FK), quantity, checked
cook_queue_entries	id (PK), user_id (FK), recipe_id (FK), added_at

Table	Columns
streaks	user_id (PK, FK users), current_streak, best_streak, cheat_days_available, last_healthy_day
user_badges	id (PK), user_id (FK), badge_name, date_earned
favorites	user_id and recipe_id (composite PK, both FK)

Notes: dates are stored as ISO text (for example 2026-10-24). Boolean flags (eaten, checked) are stored as 0 or 1. Foreign keys are switched on for every connection. All queries use PreparedStatement with ? placeholders, so user text is never joined into SQL. The complete CREATE TABLE statements are in the Database class of the project repository.

5. API Integration Design

Status: early draft (Week 9 section).

The app consumes TheMealDB (https://www.themealdb.com/api.php), a free recipe database, using its shared test key "1". The base URL used by the client is https://www.themealdb.com/api/json/v1/1/. TheMealDB returns JSON and needs no login.

Table 7: TheMealDB endpoints used

Endpoint	Purpose	Used by
search.php?s={name}	Search full recipes by name	Meal Times search and import
lookup.php?i={idMeal}	Full details of one meal by id	Opening a short-list result
filter.php?a={area}	Short list (name, photo, id) by cuisine, for example Filipino	Filipino Dishes category
list.php?a=list	List of all available cuisines	Checking which cuisines exist

Each meal in the response has the shape below. A search with no match returns "meals" as null, which the client treats as an empty list.

```
{ "meals": [ {
    "idMeal": "<id>",
    "strMeal": "<name>",
    "strCategory": "<category>",
    "strArea": "<cuisine>",
    "strInstructions": "<text>",
    "strMealThumb": "<photo url>",
    "strIngredient1": "<name>",  "strMeasure1": "<amount>",
    ... up to strIngredient20 / strMeasure20
} ] }
```

Figure 4: Shape of a TheMealDB meal response (values replaced by placeholders)

Parsing and use: the MealDbClient class sends the HTTP request, reads the JSON with Gson, and builds a Recipe object for each meal. The meal id is stored as external_id so the same recipe is never imported

twice, the photo URL appears on the Today's Meals screen, and the ingredient/measure pairs become the recipe's ingredient list used by the shopping list. TheMealDB categories such as Beef or Seafood are mapped to our three meal types (Breakfast, Dessert, otherwise MainCourse). Because the API has no nutrition data, imported dishes start as Moderate and the user chooses the Healthy Level and calories.

Additional option under consideration: a nutrition API from RapidAPI's Food APIs collection (https://rapidapi.com/collection/food-apis), such as Spoonacular, Edamam, or CalorieNinjas, to fill in calories and nutrients. It would need a personal key kept outside the repository, and results would be cached in SQLite. This option is not yet decided.

6. Concurrency Design

Status: early draft (Week 10 section).

Any operation that waits on the internet or processes a large file runs on a background thread so the interface never freezes. In JavaFX this is done with a Task started on a daemon Thread: the slow work happens in the Task's call() method, and the result is delivered through setOnSucceeded or setOnFailed, which JavaFX runs on the UI thread (the only thread allowed to change controls). While the Task runs, the screen shows a loading indicator or progress bar and stays clickable.

Table 8: Long-running operations and how they are handled

Operation	Why it is slow	Handling
TheMealDB search and import	Waits for the network	Task on a background thread; loading message; results added to the list on success; error message on failure
CSV recipe-box import	Thousands of rows to parse and save	Task with progress bar; rows saved in batches
CSV meal-plan history import	Large file with database writes	Task with progress bar
Saving queued recipes	Several database writes	Background worker
Nutrition API lookups (if adopted)	Network calls, request limits	Task, results cached in SQLite

7. Data Mining & Visualization Design

Status: early draft (Week 10 section).

The statistics step aggregates the user's meal-plan records (MealPlanEntry joined with Recipe) by counting and averaging, as listed in Table 9.

Table 9: Planned analyses

Analysis	How it is computed	Where it is shown
Meals per Healthy Level (weekly)	Count of planned meals grouped by healthy_level	Statistics: bar chart
Most-planned category and cuisine	Count grouped by meal_type and cuisine, highest wins	Statistics: text summary
Average prep time	Mean of total prep time per planned meal	Statistics: text summary
Top dishes of the week	Count grouped by recipe, top three	Statistics: list
Average calories per day	Calories of meals marked eaten divided by 7 days	Statistics: text summary
Streak	Longest and current run of consecutive days with a healthy meal eaten	Streak screen

Chart choice: a bar chart (JavaFX BarChart) shows the weekly number of Healthy, Moderate, and Indulgent meals. A bar chart suits comparing counts across a few separate categories, and it is easy to read at a glance. Both axes are labeled and the chart has a title. An optional line chart can show calories or meals over time.

8. Testing Summary

Status: early draft (Week 10 section). The tests below are written for the starter code; record the pass/fail result after running mvn test.

The JUnit 5 suite currently has 14 test cases in two classes, covering the recursion, data structures, password hashing, shopping-list merge, and streak logic. Each class has at least one edge case.

Table 10: Current test cases

Test class	Test case	What it checks
RecipeCalculatorTest	plainRecipeReturnsOwnValues	Base case: no sub-recipes
RecipeCalculatorTest	oneSubRecipeAddsUp	Prep time and calories add across one sub-recipe
RecipeCalculatorTest	threeLevelNestingIsSummedAtAnyDepth	Recursion through three levels
RecipeCalculatorTest	emptyRecipeUsesBaseEstimateAndDoesNotCrash	Edge: recipe with no ingredients and no time
RecipeCalculatorTest	subclassesEstimateDifferently	Polymorphic prep-time rules
RecipeCalculatorTest	circularLinksAreRejected	Edge: A uses B uses C, and C may not use A

Test class	Test case	What it checks
RecipeCalculatorTest	ingredientsAreCollectedFromSubRecipes	Ingredients gathered from sub-recipes
LogicTest	passwordHashIsVerifiedAndNotPlainText	Correct password passes, wrong one fails
LogicTest	undoRemovesLastActionFirst	Stack order: last in, first out
LogicTest	undoOnEmptyStackDoesNothing	Edge: undo with nothing to undo
LogicTest	cookQueueIsFirstInFirstOut	Edge: queue order and empty queue returns null
LogicTest	shoppingListMergesSameIngredientAcrossRecipes	Same ingredient, different capitalization
LogicTest	streakCountsConsecutiveHealthyDays	Edge: a gap day ends the run
LogicTest	streakSurvivesUntilEndOfTodayButBreaksAfterAMissedDay	Edge: no meal yet today, empty history

9. Individual Contributions

Status: to be completed in Week 11 from each member's own commit history.

Each member lists what they built or owned and points to their commits and files in the repository. The assigned roles are filled in from the proposal; the evidence column is completed from the git log (for example git log --author="Name" --oneline).

Table 11: Individual contributions

Member	Assigned responsibility	Built / owned (commits and files)
Kirk Quisagan	SQLite schema and JDBC data-access classes	
Ken Tamidles	TheMealDB API client and JSON parsing	
Kimberly Luistro	OOD classes, cooking queue, undo stack	
Yvonne Mae Tapan	Login/register screens and Home layout	
Venice Nadela	Calendar grid and Meal Times screen	
Nesley Cate Librea	Recursion, tests, statistics, streak, chart	

Appendix

- TheMealDB API documentation: https://www.themealdb.com/api.php
- RapidAPI Food APIs collection (nutrition API under consideration): https://rapidapi.com/collection/food-apis
- Libraries: JavaFX (GUI), SQLite JDBC driver (database), Gson (JSON parsing), JUnit 5 (testing)
- API keys, if any are adopted, are kept in a local configuration file that is excluded from the repository with .gitignore
- Course specification: Project_Specification_and_Rubric (CPE 314, Week 8)